"DAA PRACS"

191133116022

INFORMATION TECHNOLOGY [BOE 5th SEM]

INDEX

Name : TWINKLE D. SHUKLA Sub. DAA PRACTICALS

Std. : BE5 Div. : Roll No. : 191133116022

School / College :

No.	Date	Title	Pg. No.	Remark
1		PRACTICAL: 1		
		Implement following program and it's own time.		
		⋅⋅⋅ Sum of 1 to N with or without for loop	1 to 4	
2		PRACTICAL: 2		
		Implement following Sorting Algorithms	5 to 11	
		⋅⋅⋅ Insertion Sort		
		⋅⋅⋅ Bubble Sort	5	
		⋅⋅⋅ Selection Sort		
3		PRACTICAL: 3		
		Implement following Sorting Algorithms		
		⋅⋅⋅ Quick Sort	12 to 18	
		⋅⋅⋅ Merge Sort		
4		PRACTICAL: 4		
		Implement factorial knap Sack Problem Using Greedy Method.	19 to 22	

Scanned with CamScanner

5	PRACTICAL: 7 Implement 0/1 knap Sack Using Dynamic programming	23 to 25
6	PRACTICAL: 8 Solve LCS Using Dynamic programming	26 to 28
7	PRACTICAL: 9 Implement Matrix Multiplication using Strassen's Divide And Conquer Algorithm	29 to 32
8	PRACTICAL: 10 Write a program to perform N- Queens Problem	33 to 36
9	PRACTICAL: 5 Implement Dijkstra's Single Short Shortest path Algorithm.	37 to 41
10	PRACTICAL: 6 shortest Solve All pair Using Dynamic programming.	42 To 44.

Teacher's Signature.....

Scanned with CamScanner

PRACTICAL : 1

A.

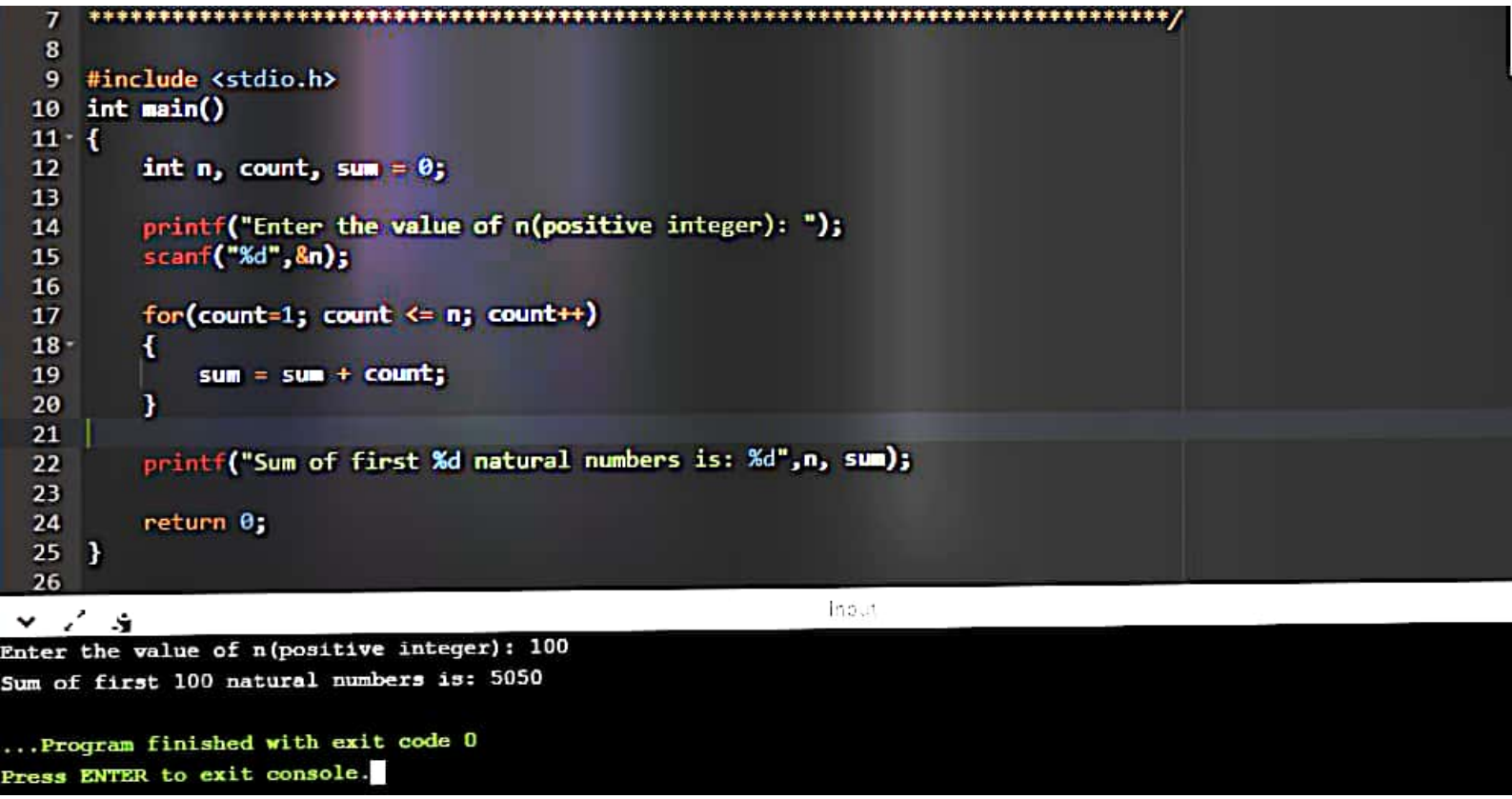
Aim: IMPLEMENT FOLLOWING PROGRAM AND ITS RUN TIME.

1. SUM OF 1 TO N NUMBER WITH AND WITHOUT FOR Loop.

WITH FOR Loop:-

```
#include<stdio.h>
{
    int main()
    {
        int n, count, sum = 0;
printf("ENTER THE VALUE OF n:");
scanf("%d", &n);
for(count = 1; count <= n; count++)
    {
        sum = sum + count;
    }
printf("SUM OF FIRST %d NATURAL
    NUMBER IS: %d", n, sum);
    return 0;
}
```

Scanned with CamScanner



```
7  ********************************************************************************/
8
9  #include <stdio.h>
10 int main()
11 {
12     int n, count, sum = 0;
13
14     printf("Enter the value of n(positive integer): ");
15     scanf("%d",&n);
16
17     for(count=1; count <= n; count++)
18     {
19         sum = sum + count;
20     }
21
22     printf("Sum of first %d natural numbers is: %d",n, sum);
23
24     return 0;
25 }
26
```

```
Enter the value of n(positive integer): 100
Sum of first 100 natural numbers is: 5050

...Program finished with exit code 0
Press ENTER to exit console.
```

Scanned with CamScanner

191133116022

Output:

```
Enter the value of n: 100
Sum of first 100 natural number is: 5050.
```

Scanned with CamScanner

Without For Loop:

```
#include<stdio.h>
int main(void){
    int n, sum;
    printf("ENTER THE VALUE OF n:");
    scanf("%d", &n);
    sum = 0;
    sum = n*(n+1)/2;
    printf("SUM: %d\n", sum);
    return 0;
}
```

Scanned with CamScanner

```c
#include <stdio.h>

int main(void) {

    int n, sum;

    //input value of n
    printf("Enter the value of n: ");
    scanf("%d", &n);

    //initialize sum with 0
    sum =0;

    //use formula to get the sum from 0 to n
    sum = n*(n+1)/2;

    //print sum
    printf("sum = %d\n", sum);

    return 0;
}
```

input

```
Enter the value of n: 100
sum = 5050


...Program finished with exit code 0
Press ENTER to exit console.
```

Scanned with CamScanner

191133116022

Date
Page 9

Output:

```
ENTER THE VALUE OF n: 100
SUM: 5050
```

Scanned with CamScanner

// REMAINDER OF THE NUMBER WITHOUT USING MODULO OPERATOR.

```
int main()
{
    int a, b, sum;
    printf("\n ENTER FIRST NO::");
    scanf("%d", &a);
    printf("\n ENTER SECOND NO::");
    scanf("%d", &b);
    sum = a - (a/b) * b;
printf("\n REMAINDER OF [%d] & [%d]
    IS = %d\n", a, b, sum);
    return 0;
}
```

Scanned with CamScanner

```c
#include <stdio.h>

int main()
{
    int a,b,rem;

    printf("\nEnter first number :: ");
    scanf("%d",&a);
    printf("\nEnter second number ::");
    scanf("%d",&b);

    rem=a-(a/b)*b;

    printf("\nRemainder of [ %d ] & [ %d ] is = %d\n",a,b,rem);

    return 0;
}
```

```
Enter first number :: 2

Enter second number ::2

Remainder of [ 2 ] & [ 2 ] is = 0
```

Scanned with CamScanner

Output:

```
ENTER FIRST NO:: 2
ENTER SECOND NO:: 2
REMAINDER OF [2] & [2] is = 0
```

Scanned with CamScanner

PRACTICAL : 2.

Aim: IMPLEMENT THE FOLLOWING SORTING ALGORITHM.

1 INSERTION SORT:

```
#include<stdio.h>
int main()
  {
    int i, j, count, temp, number[25];
printf("HOW MANY NOS YOU ARE GOING TO ENTER? : ");
scanf("%d", &count);
printf("ENTER %d ELEMENTS: ", count);
for(i=0; i< count; i++)
scanf("%d", &number[i]);
for (i=1; i< count; i++)
      {
        temp= number[i];
        j= i-1;
```

**

Scanned with CamScanner

```
while((temp<number[j])&&(j>=0)){
    number[j+1]=number[j];
    j=j-1;
}
number[j+1]=temp;}
printf("ORDER OF SORTED ELEMENTS:");
for(i=0;i<count;i++)
printf("%d",number[i]);
return 0;
```

Scanned with CamScanner

```c
7  ********************************************************************************/
8  #include<stdio.h>
9  int main(){
10      int i, j, count, temp, number[25];
11      printf("How many numbers u are going to enter?: ");
12     scanf("%d",&count);
13     printf("Enter %d elements: ", count);
14     for(i=0;i<count;i++)
15     scanf("%d",&number[i]);
16     for(i=1;i<count;i++){
17       temp=number[i];
18       j=i-1;
19       while((temp<number[j])&&(j>=0)){
20          number[j+1]=number[j];
21          j=j-1;
22       }
23       number[j+1]=temp;}
24       printf("Order of Sorted elements: ");
25       for(i=0;i<count;i++)
26       printf(" %d",number[i]);
27       return 0;}
```

input

```
How many numbers u are going to enter?: 3
Enter 3 elements: 9 10 1
Order of Sorted elements:  1 9 10

...Program finished with exit code 0
Press ENTER to exit console.
```

Scanned with CamScanner

Output:

```
How many nos you are going to enter ?: 3
Enter 3 elements: 9 10 1
Order of Sorted elements: 1 9 10
```

Scanned with CamScanner

// BUBBLE SORT:

```
#include<stdio.h>

void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;

    *yp = temp; }

void bubbleSort(int arr[], int n)
{
    int i, j;

for(i=0; i<n-1; i++)

for(j=0; j<n-i-1; j++)

 if(arr[j] > arr[j+1])

    swap(&arr[j], &arr[j+1]); }

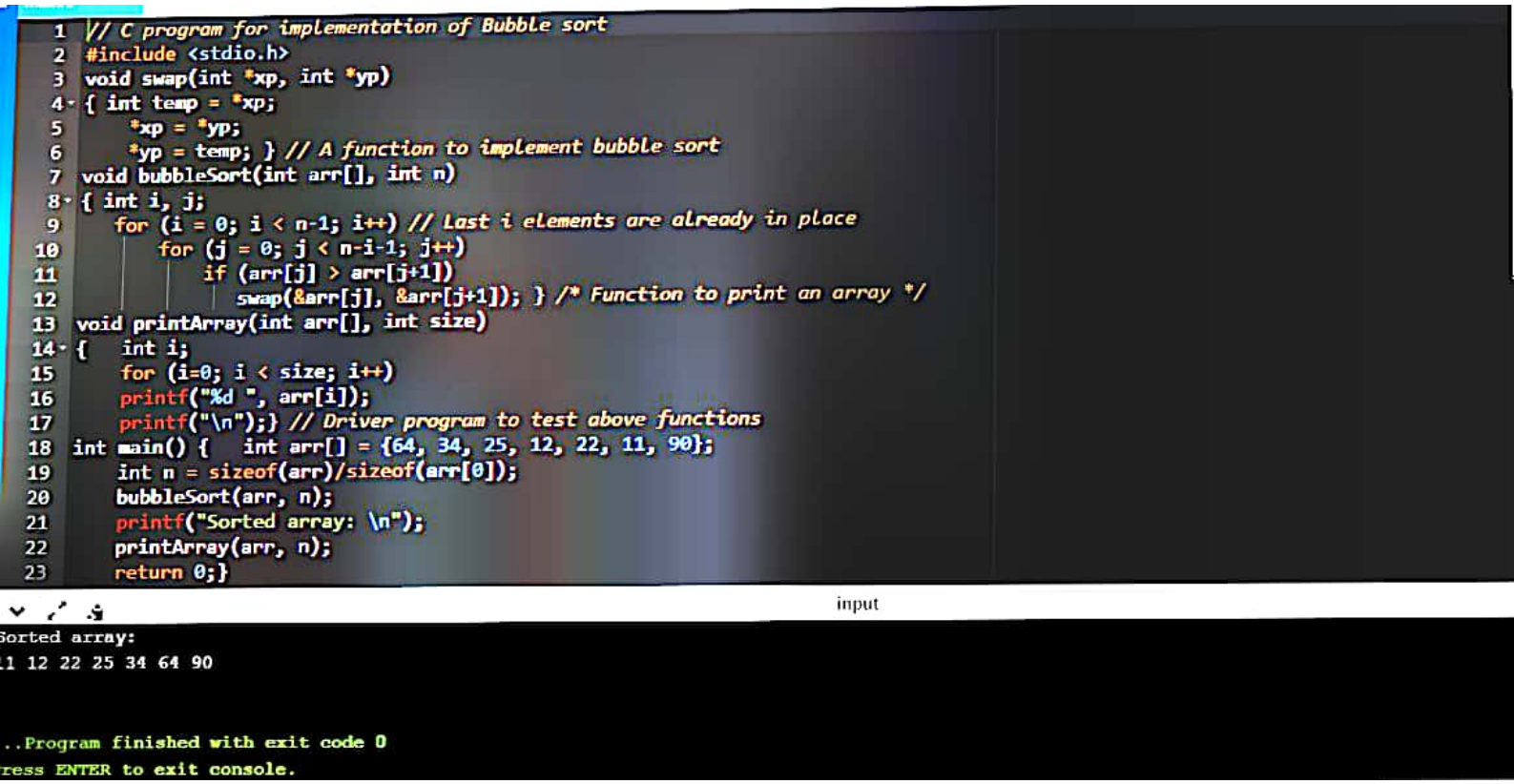
void printArray(int arr[], int size)
{
    int i;

    for(i=0; i< size; i++)
```

Scanned with CamScanner

```
printf("%d", arr[i]);
printf("\n");
int main()
{
    int arr[] = {64, 34, 25, 12, 22, 11,
                 90};
    int n = sizeof(arr)/sizeof(arr[0]);
bubbleSort(arr, n);
printf("SORTED ARRAY: \n");
printArray(arr, n);
  return 0;
}
```

Scanned with CamScanner



```c
// C program for implementation of Bubble sort
#include <stdio.h>
void swap(int *xp, int *yp)
{ int temp = *xp;
    *xp = *yp;
    *yp = temp; } // A function to implement bubble sort
void bubbleSort(int arr[], int n)
{ int i, j;
   for (i = 0; i < n-1; i++) // Last i elements are already in place
       for (j = 0; j < n-i-1; j++)
           if (arr[j] > arr[j+1])
               swap(&arr[j], &arr[j+1]); } /* Function to print an array */
void printArray(int arr[], int size)
{   int i;
    for (i=0; i < size; i++)
    printf("%d ", arr[i]);
    printf("\n");} // Driver program to test above functions
int main() {   int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0;}
```

input

```
Sorted array:
11 12 22 25 34 64 90


..Program finished with exit code 0
ress ENTER to exit console.
```

Scanned with CamScanner

Output:

SORTED ARRAY:

11 12 22 25 34 64 90

Scanned with CamScanner

III SELECTION SORT:

```
#include <stdio.h>

void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}
void selectionSort(int arr[], int n)
{
    int i, j, min_idx;
    for(i=0; i<n-1; i++)
    {
        min_idx = i;
    for(j = i+1; j<n; j++)
    if(arr[j] < arr[min_idx])
    min_idx = j;
swap(&arr[min_idx], arr[i]);
    }
}
```

Scanned with CamScanner

```
void printArray(int arr[], int size)
{
    int i;
    for(i=0; i<size; i++)
    printf("%d", arr[i]);
    printf("\n");
}

int main()
{
int arr[] = {64, 25, 12, 22, 11};
int n = sizeof(arr)/sizeof(arr[0]);
selectionSort(arr, n);
printf("SORTED ARRAY: \n");
printArray(arr, n);
return 0;
}
```

Scanned with CamScanner

```c
// C program for implementation of selection sort
#include <stdio.h>
void swap(int *xp, int *yp)
{ int temp = *xp;
    *xp = *yp;
    *yp = temp; }
  void selectionSort(int arr[], int n){  int i, j, min_idx; // One by one move boundary of unsorted subarray
    for (i = 0; i < n-1; i++){ // Find the minimum element in unsorted array
        min_idx = i;
        for (j = i+1; j < n; j++)
          if (arr[j] < arr[min_idx])
            min_idx = j; // Swap the found minimum element with the first element
        swap(&arr[min_idx], &arr[i]); } }
    void printArray(int arr[], int size) { int i;
    for (i=0; i < size; i++)
    printf("%d ", arr[i]);
    printf("\n"); } // Driver program to test above functions
    int main() { int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr, n);
    printf("Sorted array: \n");
    printArray(arr, n);
    return 0; }
```

input

```
Sorted array:
11 12 22 25 64


...Program finished with exit code 0
Press ENTER to exit console.
```

Scanned with CamScanner

191133116022

Date 11
Page

Output: SORTED ARRAY:

11 12 22 25 64.

Scanned with CamScanner

PRACTICAL : 3

AIM: IMPLEMENT THE FOLLOWING SORTING ALGORITHM.

1. QUICK SORT:-

```
#include<stdio.h>

void quicksort(int number[25], int first,
               int last)
  {
   int i, j, pivot, temp;

   if (first < last)
   {
    pivot = first;

    i = first;

    j = last;

   while (i < j)
    {
     while (number[i] <= number[pivot]
            && i < last)

     i++;
```

Scanned with CamScanner

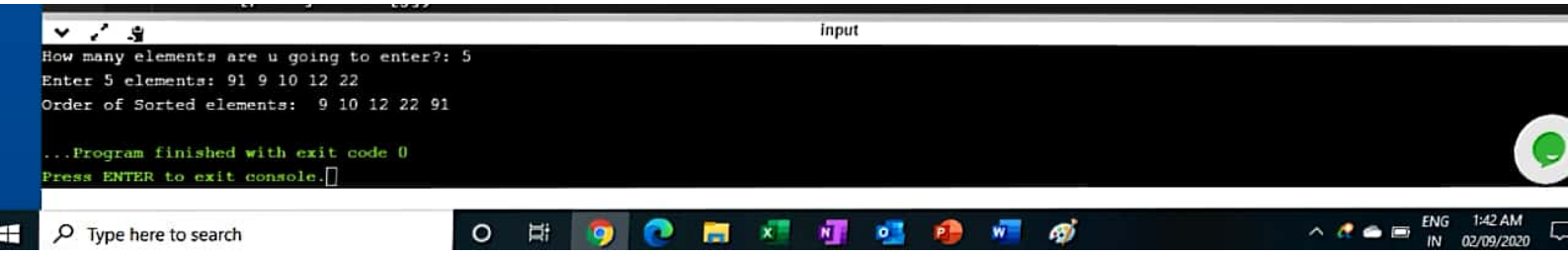
```
while(number[j] > number[pivot])
    j--;
if(i<j){
    temp = number[i];
    number[i] = number[j];
    number[j] = temp;
}
}

temp = number[pivot];
number[pivot] = number[j];
number[j] = temp;
quicksort(number, first, j-1);
quicksort(number, j+1, last);
}
}
int main(){
    int i, count, number[25];
printf("How many elements you are going to enter:");
```

Scanned with CamScanner

```
scanf("%d", &count);
printf("ENTER %d ELEMENTS:", count);
for(i=0; i<count; i++)
scanf("%d", &number[i]);
quickSort(number, 0, count-1);
printf("ORDER OF SORTED ELEMENTS:");
for(i=0; i<count; i++)
printf("%d", number[i]);
return 0;
}
```

Scanned with CamScanner



Scanned with CamScanner

```
printf("Enter %d Elements:", count);
for(i=0; i<count; i++)
scanf("%d", &number[i]);
quickSort(number, 0, count-1);
printf("Order Of Sorted Elements:");
for(i=0; i<count; i++)
printf("%d", number[i]);
return 0;
}
```

Output:

```
How Many Elements You Are Going To Enter ?: 5
Enter 5 Elements: 91 9 10 12 22
Order Of Sorted Elements:
9 10 12 22 91
```

Scanned with CamScanner

191133116022

Date 15
Page

11. MERGE SORT:-

```
#include<stdio.h>
void mergeSort(int [], int, int, int);
void partition(int [], int, int);
int main()
{
  int list[50];
  int i, size;
  printf("ENTER TOTAL NO. OF ELEMENTS:");
  scanf("%d", &size);
  printf("ENTER THE ELEMENTS:\n");
  for(i=0; i<size; i++)
  {
    scanf("%d", &list[i]);
  }
  partition(list, 0, size-1);
  printf("AFTER MERGE SORT:\n");
  for(i=0; i<size; i++)
  {
  printf("%d", list[i]);
  }
```

Scanned with CamScanner

```
    return 0;
}
void partition (int list[], int low,
                int high)
  {
    int mid;
if( low < high)
  {
   mid = (low + high) /2;

partition(list, low, mid);
partition( list, mid+1, high);
mergeSort( list, low, mid, high);
    }
}
Void mergeSort( int list[], int low,
  int mid, int high)
  {
    int i, mi, k, lo, temp[50];

    lo = low;
    i = low;
   mi = mid + 1;
while((lo <= mid) && (mi <= high))
    {
      if (list[lo] <= list[mi])
        {
          temp[i] = list[lo];
          lo++;
```

Scanned with CamScanner

```
    else
    {
        temp[i] = list[mi];
        mi++;
    }
    i++;
}
if( lo > mid)
{
for(k= mi; k<= high ; k++)
    {
        temp[i]= list[k];
        i++;
    }
}

else
{
for(k= lo; k <= mid; k++)
    {
        temp[i] = list[k];
        i++;
    }
}

for(k= low; k <= high; k++)
    {
        list[k]= temp[k];
    }
}
```

Scanned with CamScanner

```
Enter total number of elements:3
Enter the elements:
91 5 22
After merge sort:
5   22   91
```


Scanned with CamScanner

Output:

```
ENTER TOTAL NUMBER OF ELEMENTS:3
ENTER THE ELEMENTS:
91 5 22
AFTER MERGE SORT:
5 22 91
```

Name: f

#

Scanned with CamScanner

191133116022

Date 19
Page

PRACTICAL : 4.

AIM: IMPLEMENT FACTORIAL KNAP SACK PROBLEM USING GREEDY METHOD.

```
#include <stdio.h>

void knapsack(int n, float weight[], float profit[], float capacity)
{
  float x[20], tp=0;
  int i, j, u;

  u= capacity;

  for(i=0; i<n; i++)
    x[i]= 0.0;

  for(i=0; i<n; i++)
  {
    if(weight[i]> u)
      break;

    else
    {
      x[i]= 1.0;
      tp= tp+ profit[i];

      u= u- weight[i];
    }
  }
```

Scanned with CamScanner

```
if(i<n)
    x[i]= u/weight[i];
tp= tp+(x[i]* profit[i]);
printf("\n THE RESULT VECTOR IS:");
for(i=0; i<n; i++)
printf("%f\t", x[i]);
printf("\n MAXIMUM PROFIT IS: %f",
                tp);
}

int main()
{
float weight[20], profit[20], capacity;
int num, i, j;
float ratio[20], temp;
printf("\n ENTER NO. OF OBJECTS:");
scanf("%d", &num);
```

Scanned with CamScanner

Date Page 29

```
printf("\n ENTER THE WEIGHTS AND PROFITS OF EACH OBJECT: ");

for (i=0; i<num; i++)
{
    scanf("%f %f", &weight[i], &profit[i]);
}

printf("\n ENTER THE CAPACITY OF KNAPSACK: ");
scanf("%d", &capacity);

for(i=0; i<num; i++)
{
    ratio[i]= profit[i] / weight[i];
}

for(i=0; i<num; i++)
{
    for(j= i+1; j<num; j++)
    {
        if(ratio[i]< ratio[j])
        {
            temp = ratio[j];
            ratio[j]= ratio[i];
            ratio[i]= temp;
            temp= weight[j];
```

Scanned with CamScanner

```
        weight[j]= weight[i];
        weight[i]= temp;
        temp= profit[j];
        profit[j]= profit[i];
        profit[i]= temp;
    }
}
knapsack(num, weight, profit, capacity);
    return(0);
}
```

Scanned with CamScanner

```
Enter the no. of objects:- 4

Enter the wts and profits of each object:- 40 280 10 100 20 120 24 120

Enter the capacityacity of knapsack:- 60

The result vector is:- 1.000000 1.000000        0.500000        0.000000
Maximum profit is:- 440.000000

...Program finished with exit code 0
Press ENTER to exit console.
```

Scanned with CamScanner

Output:

```
Enter the No. of Objects: 4
Enter the Weights and Profit of each Objects:
40 200 10 100 20 120 24 120
Enter the Capacity of knapsack: 60
The Result Vector is:
1.00000 1.00000 0.500000 0.00000
Maximum Profit is: 440.00000
```

Scanned with CamScanner

191133116022
3rd

PRACTICAL 5.

Aim: IMPLEMENT DIJKSTRA'S SINGLE SHORT PATH ALGORITHM.

```
#include <stdio.h>
#include <conio.h>
#define INFINITY 9999
#define MAX 10

void dijkstra(int G[MAX][MAX],
              int n, int startnode);

int main()
{
    int G[MAX][MAX], i, j, n, u;

    printf("ENTER NO. OF VERTICES:");
    scanf("%d", &n);

    printf("\n ENTER ADJACENCY MATRIX:\n");

    for(i=0; i<n; i++)

        for(j=0; j<n; j++)
            scanf("%d", &G[i][j]);

    printf("\n ENTER STARTING NODE:");

    scanf("%d", &u);
    dijkstra(G, n, u);
    return 0;
```


Scanned with CamScanner

```
void dijkstra(int G[MAX][MAX],
{
int cost[MAX][MAX],
    distance[MAX], pred[MAX];

int visited[MAX], count, mindistance,
nextnode, i, j;

for(i=0; i<n; i++)
        for(j=0; j<n; j++)

    if(G[i][j]==0)
        cost[i][j]= INFINITY;
    else
        cost[i][j]= G[i][j];

for(i=0; i<n; i++)
{
    distance[i] = cost[startnode][i];
    pred[i]= startnode;

    visited[i]=0;
}

distance[startnode]=0;
visited[startnode]=1;
count = 1;
```

Scanned with CamScanner

```
while (count<n-1)
{
  mindistance= INFINITY;
  for(i=0;i<n;i++)
    if (distance[i]< mindistance
    && !visited[i])
{
  mindistance = distance[i];
  nextnode = i;
}

  visited[nextnode] = 1;
  for(i=0; i<n; i++)
    if(!visited[i])
      if(mindistance + cost[nextnode]
        [i]< distance[i])
  {
    distance[i]= mindistance +
        cost[nextnode][i];
  }
  count++;
}
  for(i=0;i<n;i++)
    if(i!=startnode)
  {
    printf("\n DISTANCE OF node %d
        = %d", i, distance[i]);
    printf("\n PATH =%d", i);
```

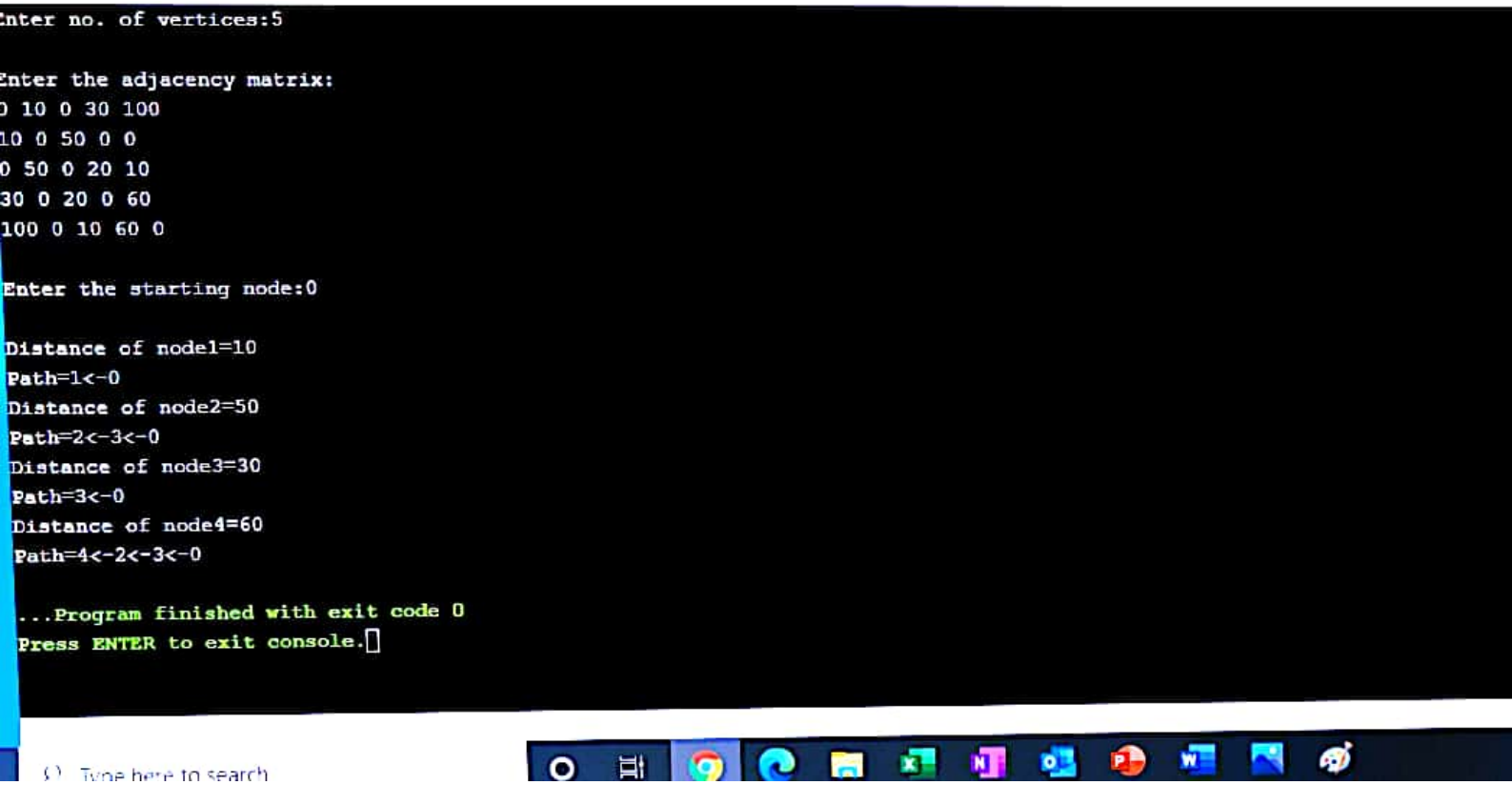
Scanned with CamScanner

```
        j=i;
    {   do
        j=pred[j];
        printf("<-%d", j);
    }
    while(j!=startnode);
    }
}
```

Output:

```
ENTER NO. OF VERTICES: 5
ENTER THE ADJACENCY MATRIX:
0   10   0   30   100
10   0   50   0   0
0   50   0   20   10
30   0   20   0   60
```

Scanned with CamScanner



Scanned with CamScanner

```
        j=i;
        {
        do
        {
            j=pred[j];
            printf("<-%d",j);
        }
        while(j!=startnode);
        }
}
```

Output:

```
ENTER NO. OF VERTICES: 5
ENTER THE ADJACENCY MATRIX:
0   10   0   30   100
10   0   50   0   0
0   50   0   20   10
30   0   20   0   60
100   0   10   60   0
```

Scanned with CamScanner

ENTER THE STARTING NODE: 0

DISTANCE OF NODE 1 = 10
PATH = 1 <= 0

DISTANCE OF NODE 2 = 50
PATH = 2 ← 3 ← 0

DISTANCE OF NODE 3 = 30
PATH = 3 ← 0

DISTANCE OF NODE 4 = 60
PATH = 4 ← 2 ← 3 ← 0.

Scanned with CamScanner

Date 99
Page

PRACTICAL : 6

AIM: SOLVE ALL PAIR SHORTEST PATH USING DYNAMIC PROGRAMMING.

```
#include<iostream>
using namespace std;
#define nV 4
#define INF 999
void printMatrix(int matrix[][nV]);
void floydWarshall(int graph[][nV])
{
    int matrix[nV][nV], i, j, k;
    for(i=0; i<nV; i++)
        for(j=0; j<nV; j++)
            matrix[i][j]= graph[i][j];
    for(k=0; k<nV; k++) {
        for(i=0; i<nV; i++) {
            for(j=0; j<nV; j++) {
```

Scanned with CamScanner

```
        if(matrix[i][k]+ matrix[k][j]<
                matrix[i][j])
            matrix[i][j]= matrix[i][k]+
                    matrix[k][j];
        }
    }
}
    printMatrix(matrix);
}
void printMatrix(int matrix[][nV])
    {
    for(int i=0; i<nV; i++) {
      for(int j=0; j<nV; j++) {
    if(matrix[i][j]== INF)
      printf("%4s", "INF");
    else
    printf("%4d", matrix[i][j]);
    }
      printf("\n");
  }
}
```

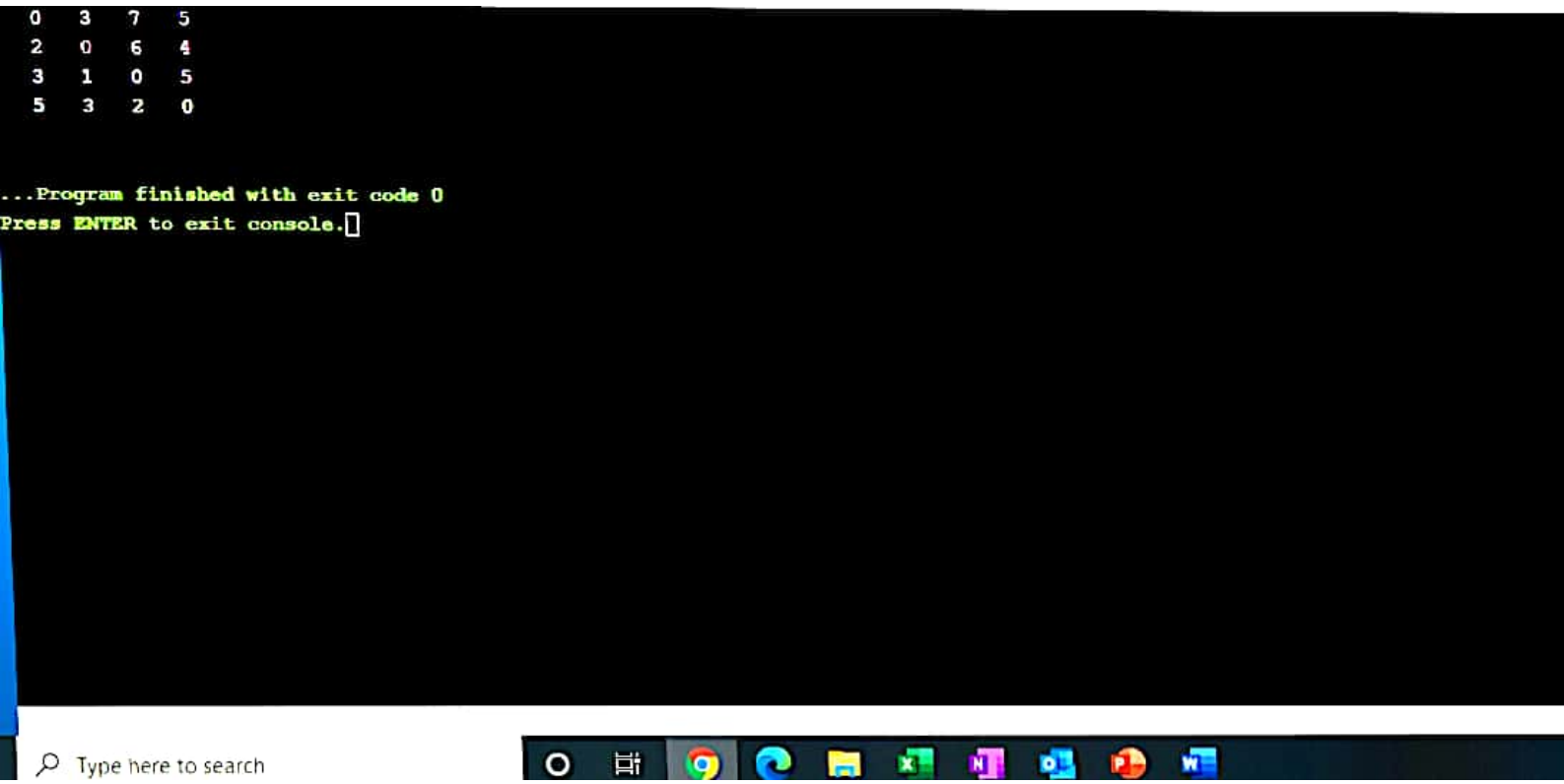
Scanned with CamScanner

Date 44
Page

```
int main () {
int graph[nV][nV]={{0, 3, INF, 5},
{2, 0, INF, 4},
{INF, 1, 0, INF},
{INF, INF, 2, 0}};

floydWarshall(graph);
}
```

Scanned with CamScanner



Scanned with CamScanner

Output:

```
0   3   7   5
2   0   6   4
3   1   0   5
5   3   2   0
```

Scanned with CamScanner

191133116022

Date 23
Page

PRACTICAL : 7

AIM: IMPLEMENT 0/1 KNAPSACK USING DYNAMIC PROGRAMMING.

```
#include <stdio.h>

int max(int a, int b)
{
return (a > b) ? a : b;
}
int knapSack(int W, int wt[], int val[],
             int n)
{
  int i, w;
int K[n+1][W+1];

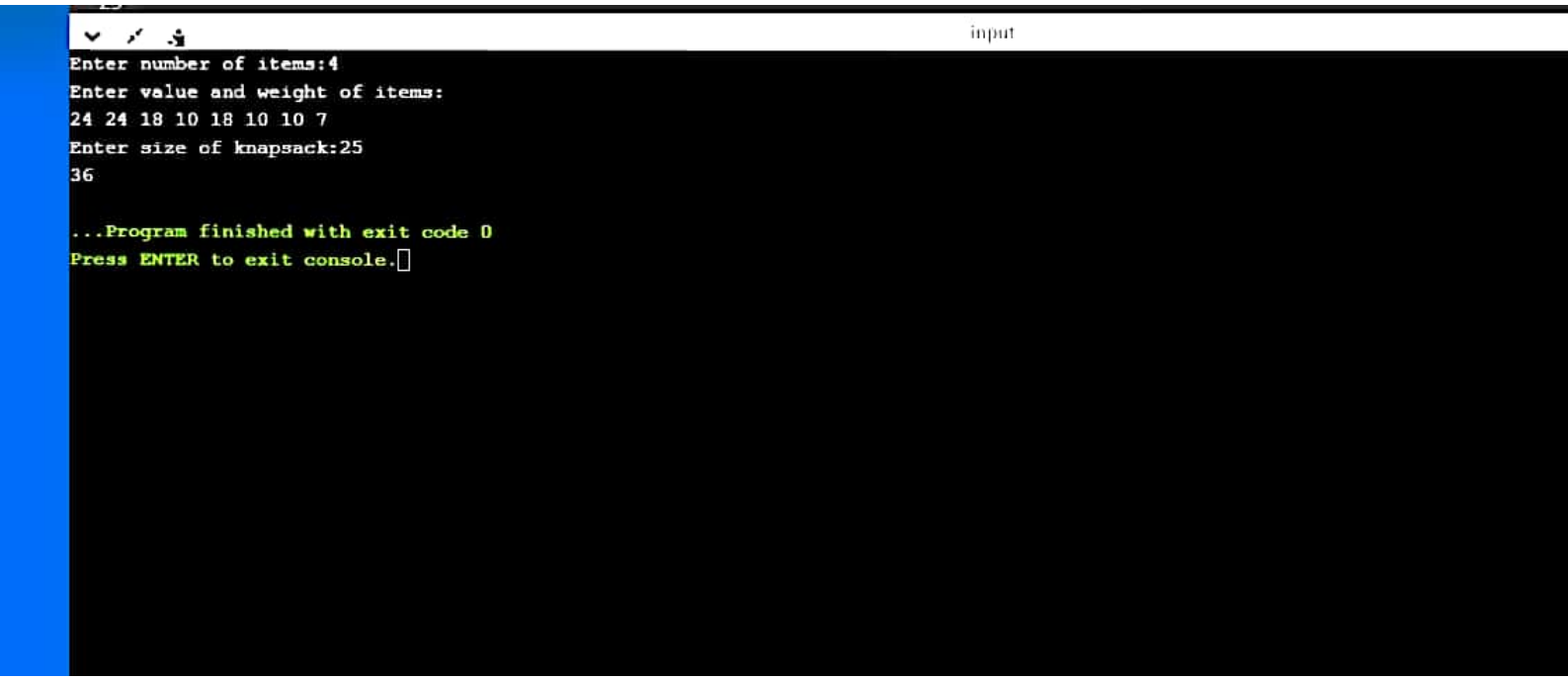
for(i=0; i<=n; i++)
{
  for(w=0; w<=W; w++)
{
if(i==0 || w==0)
K[i][w]=0;
else if(wt[i-1]<=w)
K[i][w]= max(val[i-1]+K[i-1]
[w-wt[i-1]], K[i-1][w]);
```

Scanned with CamScanner

```
            else
            K[i][w] = K[i-1][w];
        }
    }
    return K[n][W];
}

int main()
{
    int i, n, val[20], wt[20], W;
    printf("ENTER NO. OF ITEMS:");
    scanf("%d", &n);
    printf("ENTER VALUE AND WEIGHT
        OF ITEMS:\n");
    for(i=0; i<n; ++i)
    {
        scanf("%d %d", &val[i], &wt[i]);
    }
    printf("ENTER SIZE OF KNAP-SACK:");
    scanf("%d", &W);
    printf("%d", knapSack(W, wt, val, n));
    return 0;
}
```

Scanned with CamScanner



Scanned with CamScanner

191133116022

Date 25

Output: ENTER NO. OF ITEMS: 4

ENTER VALUE AND WEIGHT OF ITEMS:

24 24 18 10 18 10 10 7

ENTER SIZE OF KNAPSACK: 25

36.

Scanned with CamScanner

PRACTICAL : 8

Aim: Explain / Solve Longest Common Subsequence Problem Using Dynamic Method.

```
#include<stdio.h>
#include<string.h>

int i, j, m, n, c[20][20];
char x[20], y[20], b[20][20];

void print(int i, int j)
{
    if(i==0 || j==0)
        return;

    if(b[i][j] == 'c')
    {
    print(i-1, j-1);
    printf("%c", x[i-1]);
    }

    else if(b[i][j] == 'u')
        print(i-1, j);
    else
```

Scanned with CamScanner

```
else
    {
        c[i][j]= c[i][j-1];
        b[i][j]= 'l';
    }
}
}
int main()
{
    printf("ENTER 1st SEQUENCE:");
    scanf("%s", x);
    printf("ENTER 2nd SEQUENCE:");
    scanf("%s", y);
    printf("\n THE LONGEST COMMON
SUB SEQUENCE IS:");
    lcs();
    print(m, n);
    return 0;
}
```

Scanned with CamScanner

input

```
Enter 1st sequence:abcdbacdf
Enter 2nd sequence:cbaf

The Longest Common Subsequence is cbaf

...Program finished with exit code 0
```



Scanned with CamScanner

```
        b[i][j] = 'l';
    }
}
int main()
{
    printf("ENTER 1ST SEQUENCE: ");
    scanf("%s", x);
    printf("ENTER 2nd SEQUENCE: ");
    scanf("%s", y);
    printf("\n THE LONGEST COMMON
SUB SEQUENCE IS: ");
    lcs();
    print(m, n);
    return 0;
}
```

Output:

```
ENTER 1ST SEQUENCE: abcd bacdf
ENTER 2nd SEQUENCE: cbaf
THE LONGEST COMMON SUB SEQUENCE IS:
cbaf
```

Scanned with CamScanner

191133116022

99

PRACTICAL 9.

Aim: Implement Matrix Multiplication using Strassen's Divide and Conquer Algorithm.

```
#include <stdio.h>
int main()
{
int a[2][2], b[2][2], c[2][2], i, j;
int m1, m2, m3, m4, m5, m6, m7;
printf("Enter 4 Elemets of 1st Matrix:");
for(i=0; i<2; i++)
    for(j=0; j<2; j++)
        scanf("%d", &a[i][j]);
printf("Enter 4 Elements of 2nd Matrix:");
for(i=0; i<2; i++)
    for(j=0; j<2; j++)
        scanf("%d", &b[i][j]);
printf("\n The First Matrix is\n");
```

Scanned with CamScanner

```
for(i=0; i<2; i++)
{
  printf("\n");
  for(j=0; j<2; j++)
  printf("%d\t", a[i][j]);
}

printf("\n THE SECOND MATRIX IS \n");
  for(i=0; i<2; i++)
  {
    printf("\n");
  for(j=0; j<2; j++)
    printf("%d\t", b[i][j]);
  }

m1=(a[0][0]+ a[1][1])*
      (b[0][0]+ b[1][1]);

m2=(a[1][0]+ a[1][1])*
      b[0][0];

m3= a[0][0] * b[0][1]- b[1][1];
```

Scanned with CamScanner

```
m4= a[1][1]*(b[1][0]- b[0][0]);

m5 =(a[0][0]+ a[0][1])* b[1][1];

m6= (a[1][0] - a[0][0])*
                (b[0][0] + b[0][1]);

m7= (a[0][1] - a[1][1])*
                (b[1][0]+ b[1][1]);

C[0][0]= m1+ m4-m5+m7;

C[0][1]= m3 + m5;

C[1][0] = m2+m4;

C[1][1]= m1- m2 +m3+m6;

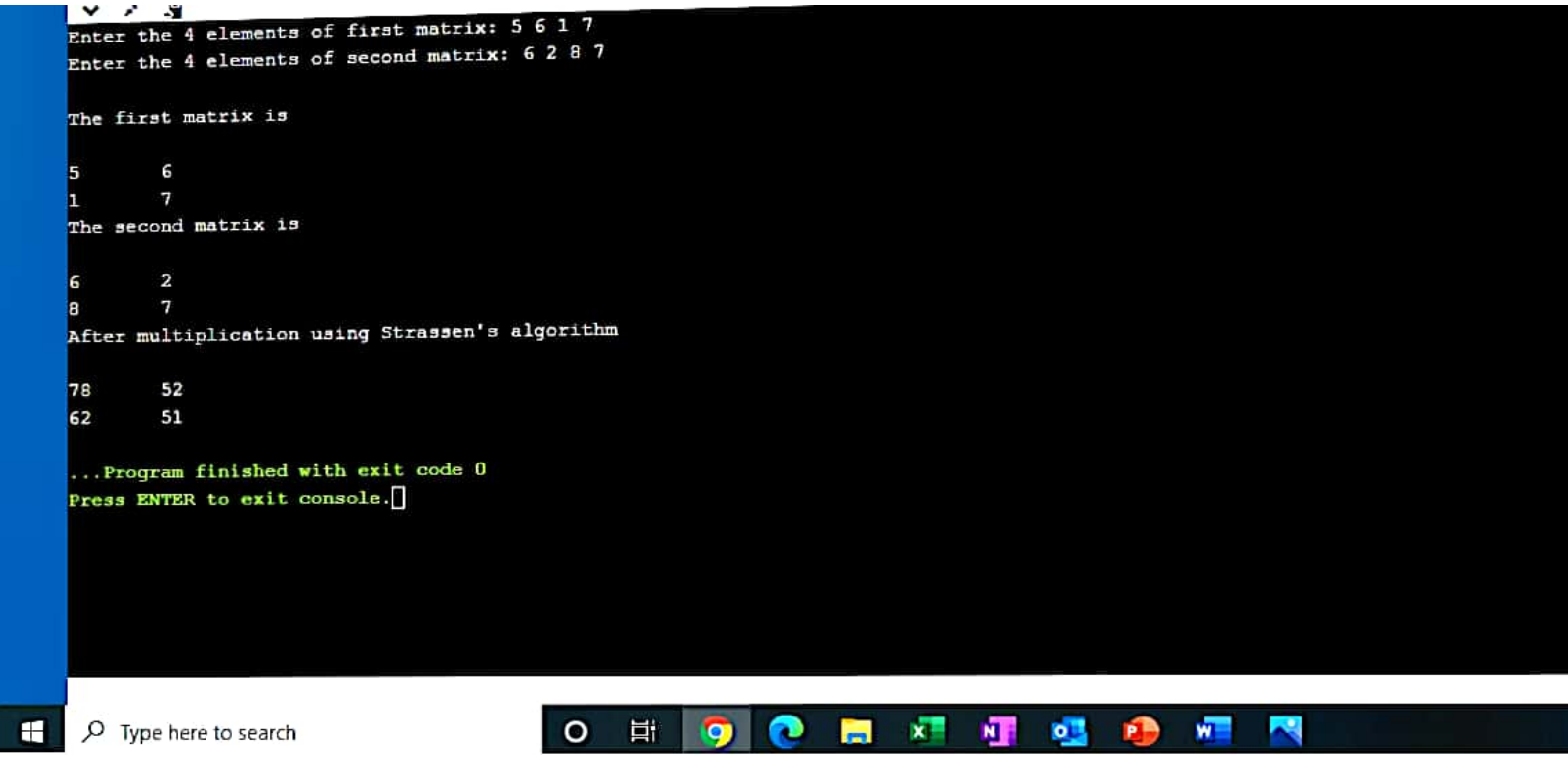
printf("\n AFTER MULTIPLICATION USING \n");
                                STRASSEN'S ALGORITHM
 for(i=0; i<2; i++)
      {
         printf("\n");

      for(j=0; j<2; j++)

      printf("%d\t", C[i][j]);
      }

      return 0;
}
```

Scanned with CamScanner



Scanned with CamScanner

Output: Enter the 4 Elements of
First Matrix is: 5 6 1 7

Enter the 4 Elements of
Second Matrix is: 6 9 8 7

The First Matrix is.

5 6

1 7

The Second Matrix is

6 9

8 7

After Multiplication Using
Strassen's Algorithm

78 59

69 51

Aim: Wri

#in
#in
#in

int

in

for

Scanned with CamScanner

1911133116022

33

PRACTICAL : 10

AIM: WRITE A PROGRAM TO PERFORM N QUEEN PROBLEM.

```
#include <stdio.h>
#include <conio.h>
#include <math.h>

int board[20], count;

int main()
{
    int n, i, j;
    void queen(int row, int n);
printf("- N QUEENS PROBLEM
        USING BACKTRACKING -");
printf("\n\n ENTER NUMBER OF
        QUEENS :");
scanf("%d", &n);
queen(1, n);
  return 0;
}
```

Scanned with CamScanner

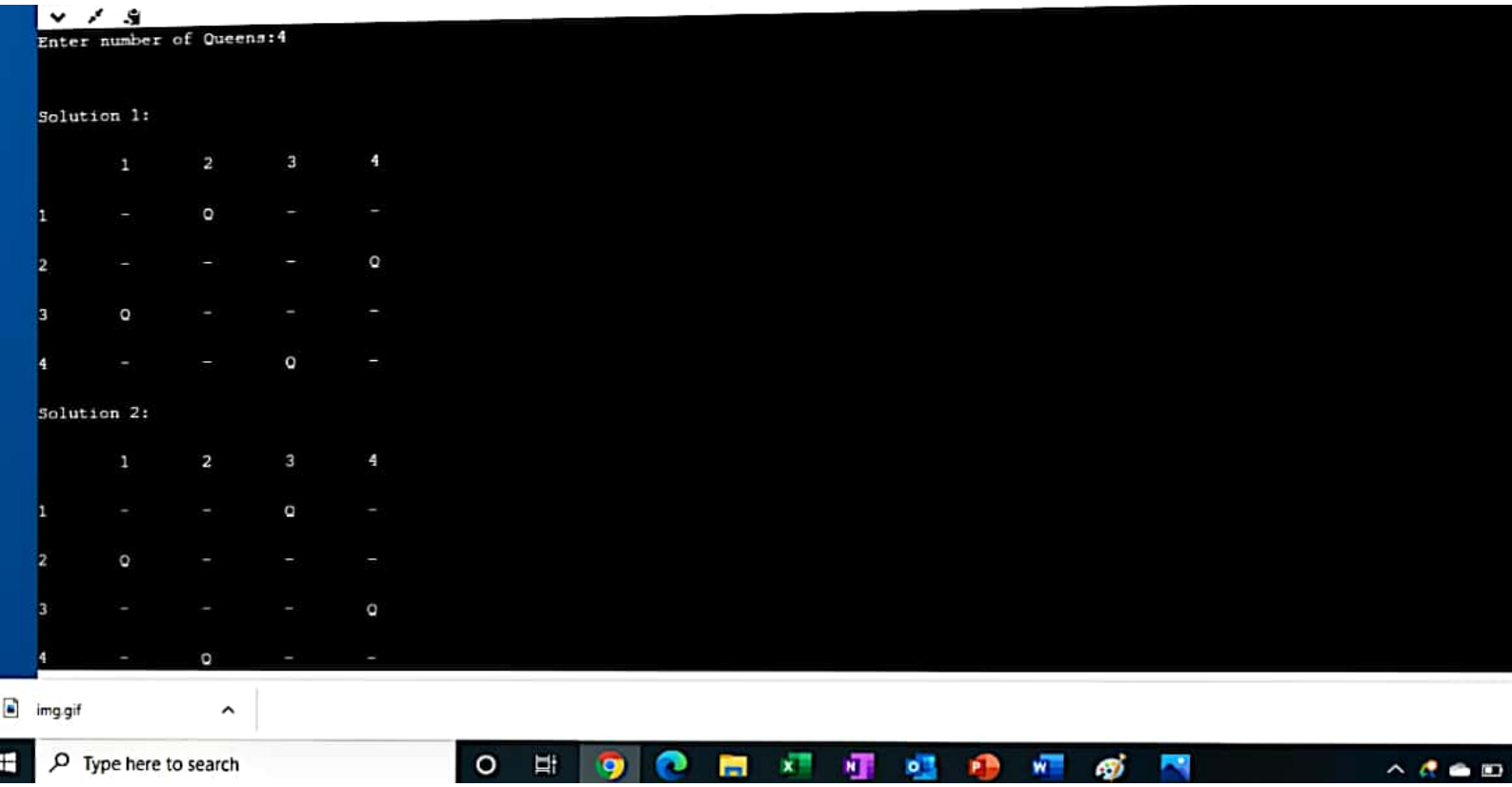
```
void print(int n)
{
    int i, j;
printf("\n\n Solution %d:\n\n",
                ++count);
    for (i=1; i<=n; ++i)
    printf("\t %d", i);
    for (i=1; i<=n; ++i)
    {
    printf("\n\n %d", i);
    for(j=1; j<=n; ++j) //for nxn
                        board.
    {
      if (board[i]== j)
      printf("\t@"); //queen @
                        i, j pos.
      else
      printf("\t-"); //empty slot
    }
  }
}
```

Scanned with CamScanner

```
int place (int row, int column)
{
 int i;
 for(i=1; i<= row- 1; ++i)
  {
 if (board[i]== column)
  return 0;
 else
 if (abs( board[i]- column )== abs (i-row))
  return 0 ;
 }
  return 1;
}
void queen (int row, int n)
  {
    int column;
for( column=1; column <= n; ++ column)
  {
    if(place(row, column))
{
 board[row]= column;
 if(row==n)
```

Scanned with CamScanner

```
        print(n);
    else
        queen(row+1, n);
    }
}
```



Scanned with CamScanner

y

Output: ENTER NUMBER OF QUEENS: 4

SOLUTION IS:

	1	2	3	4
1	-	Q	-	-
2	-	-	-	Q
3	Q	-	-	-
4	-	-	Q	-

Scanned with CamScanner